

Server farms

Server farms



Server farms

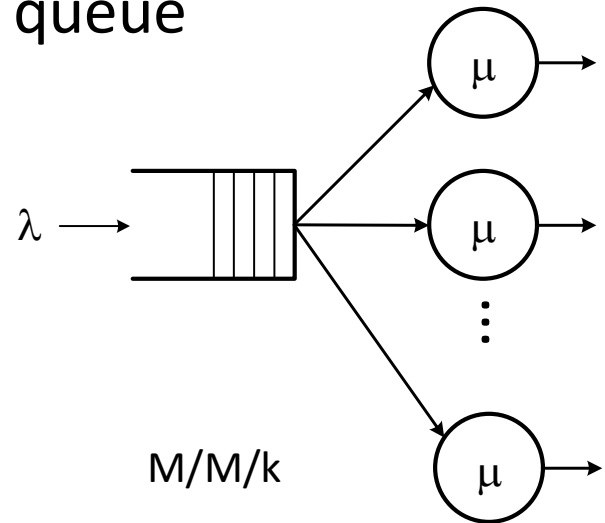
- In today's "high-volume world", almost no websites, computer centers, or call centers consist of just a single server.
- Server farms = collection of servers that work together to handle incoming requests; each request may be routed to a different server; datacenters are made of server farms.
- Preferable to single super-fast server, due to lower cost and higher flexibility.
- Two possible arrangements:
 - Central queue
 - Load balancer (dispatcher)



Server farms with central queue

$$\frac{W^{M/M/k}}{W^{M/M/1}} = P_Q^{M/M/k} + k(1 - \rho)$$

- $\rho \approx 0$: M/M/1 is k times faster
- $\rho \approx 1$: M/M/1 and M/M/k have similar performance



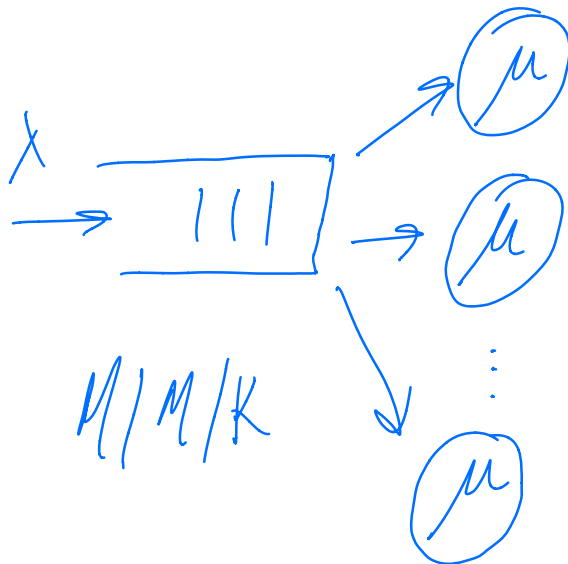
M/M/k



M/M/1



$$= \frac{\frac{\lambda/\kappa\mu}{\lambda(1 - \lambda/\kappa\mu)}}{\lambda(1 - \rho)} = \frac{\rho}{\lambda(1 - \rho)}$$



$$W^{M/M/k} = \frac{1}{\lambda} \cdot P_0 \cdot \frac{\rho}{1 - \rho} + \frac{1}{\mu}$$

$$\frac{W^{M/M/K}}{W^{M/M/1}} = \frac{\frac{1}{\lambda} \cdot \rho_Q \cdot \frac{\rho}{1-\rho} + \frac{1}{\mu}}{\frac{1}{\lambda} \frac{\rho}{1-\rho}}$$

$$= \rho_Q + \frac{\lambda}{\mu} \frac{1-\rho}{\rho} = \rho_Q + \kappa(1-\rho)$$

$$\frac{\lambda}{\mu \rho} = \frac{\cancel{\lambda} \kappa \cancel{\mu}}{\cancel{\mu} \cancel{\lambda}} = \kappa$$

$$\frac{W^{M/M/K}}{W^{M/M/1}} = \rho_Q^{M/M/K} + K(1-\rho)$$

$$\rho \approx 0 \Rightarrow \rho_Q \approx 0 \Rightarrow \frac{W^{M/M/K}}{W^{M/M/1}} \approx K$$

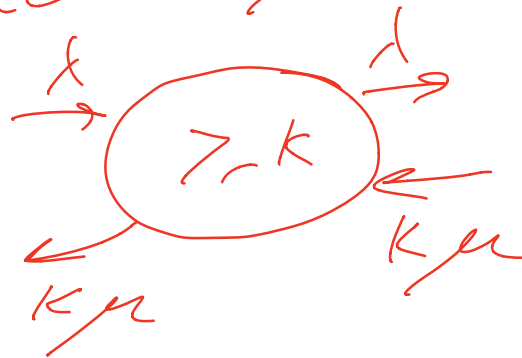
$M/M/K$ most servers idle; servers
 busy serve at rate μ
 $M/M/1$ the few jobs are served
 at rate $K\mu$

$$\frac{W^{M/M/K}}{W^{M/M/1}} = \rho_q^{M/M/K} + K(1-\rho)$$

$$\rho \approx 1 \Rightarrow \rho_q \approx 1 \Rightarrow$$

$$\frac{W^{M/M/K}}{W^{M/M/1}} \approx 1 + K - \rho K \approx 1$$

Always jobs in queue
 $M/M/K$ states $\geq K$ are
 more probable $\rightarrow$ similar
 to $M/M/1$



Capacity provisioning for M/M/k server farms

- If $R = \lambda/\mu$ is large, the minimum number of servers to get $P_Q < \alpha$ is smallest integer equal or larger than

$$k_{\alpha}^* \approx R + c\sqrt{R}$$

$\alpha = 0.8$	$\alpha = 0.5$	$\alpha = 0.2$	$\alpha = 0.1$
$c = 0.173$	$c = 0.506$	$c = 1.06$	$c = 1.42$

- A datacenter has to handle jobs arriving at a rate of $\lambda = 5$ jobs/sec with average service duration $1/\mu = 1$ second. The service time is exponentially distributed and the job arrival process is Poisson. What is the minimum numbers of servers required to ensure that $P_Q < 0.2$?

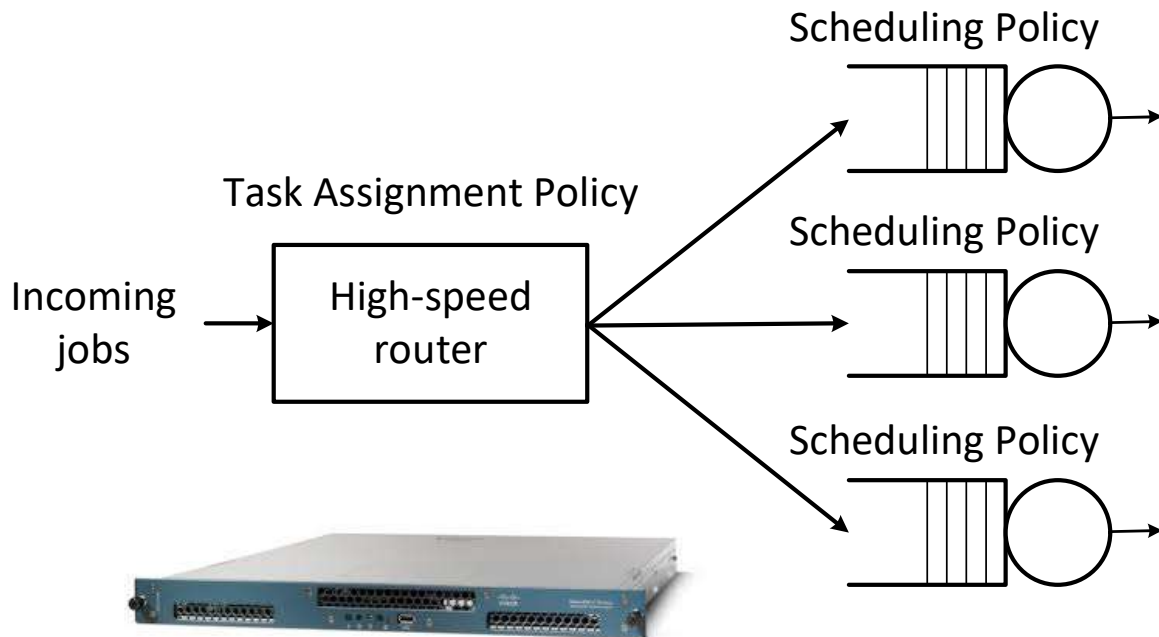
$$R = \lambda/\mu = 5 \quad k_{0.2}^* \approx 5 + \sqrt{5} = 7.2 \Rightarrow 8 \text{ servers}$$

Capacity provisioning for M/M/k server farms

- Approximation accuracy ($\alpha = 0.2$)

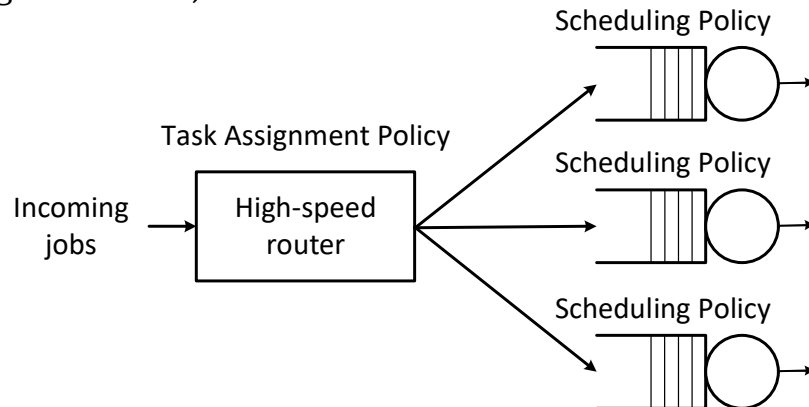
	Exact $k_{0.2}^*$	Approximate $k_{0.2}^*$
$R = 1$	3	2
$R = 5$	8	8
$R = 10$	14	14
$R = 50$	58	58
$R = 100$	111	111
$R = 250$	268	267
$R = 500$	525	524
$R = 1000$	1034	1034

Server farm with load balancer (dispatcher)



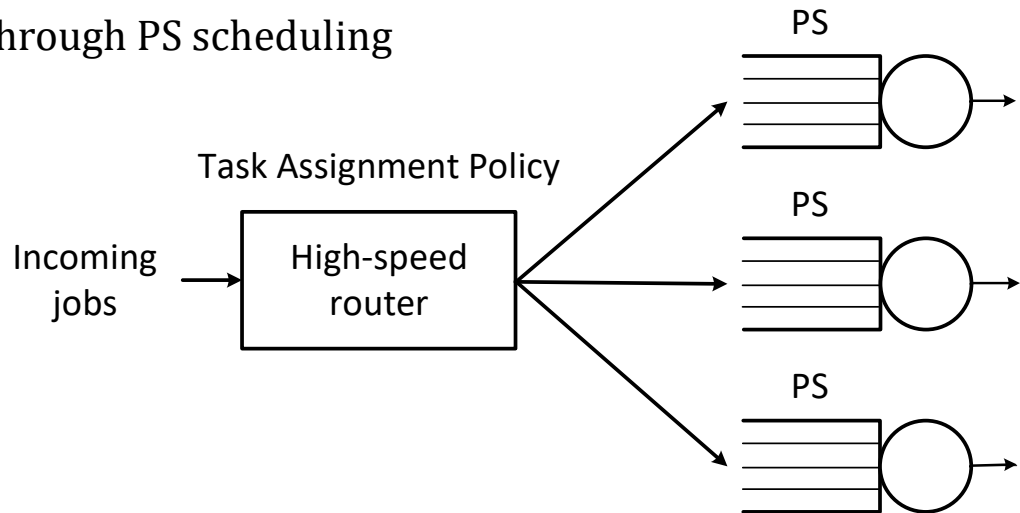
Server farm with load balancer (dispatcher)

- Task assignment policies:
 - Random – each job is assigned to one of k hosts with equal probability
 - Round-robin – the i -th job is assigned to host $(i \bmod k) + 1$
 - JSQ (Join-the-Shortest-Queue) – each job is assigned to host with fewer queued jobs
 - LWL (Least-Work-Left) – each job is assigned to host with least total work
 - SITA (Size-Interval-Task-Assignment) – small jobs go to host 1, mediums to host 2, and larges to host 3, etc.



Web server farms

- Incoming jobs are HTTP requests; need to be dispatched immediately
- Request cannot be queued; each host time-shares among all requests in queue so that HTTP request receives “constant” service
- Modeled through PS scheduling





Web server farms

- Average delay in system with **random** assignment policy, k servers, Poisson arrivals at rate λ and service rate μ . Each server is M/G/1/PS = M/M/1, thus

$$W = \frac{1}{\mu - \lambda/k}$$

- JSQ has better performance, but no available theoretical analysis

